Homework 1: Image Enhancement

Computer Vision, Fall 2026 · Mattia Bortolaso
Code and results: https://github.com/Borto71

1. Objective

The goal of this homework is to build a small pipeline of classical image processing methods to improve colour images that suffer from low light, low contrast and noise, and to compare these methods both with numbers and by looking at the results. The aim is not to perfectly recover the scene, but to understand what each method actually does to the image, where it helps and what it breaks. Everything is done with OpenCV and NumPy, no learning-based method is used.

2. Images and degradation

I used five colour photos (Table 1): one of my own and four from Pexels. All of them are real scenes with at least one of the problems of interest: fog and low contrast (img1, img3), strong backlight with almost black areas (img2, img5), or local lamps next to deep shadows (img4). All the images are larger than 1024 px on the longest side, so they are resized to that size with `INTER_AREA`.

image	content	condition	source	license
img1	Foggy street at night, seen from a window	low light, fog, light sources	Personal photo	Own work
img2	City skyline in silhouette at sunset	backlight, very dark buildings	Pexels, photo 18927434, Oscar Portan	Pexels License
img3	Foggy road with bare trees	low contrast, fog	Pexels, photo 34649347, Christina & Peter	Pexels License
img4	Night market fruit stall under lamps	non-uniform light, dark shadows	Pexels, photo 34559646, Jahra Tasfia Reza	Pexels License
img5	Dark living room, backlit by a window	non-uniform light, dark shadows	Pexels, photo 30422461, Try Putro Utomo	Pexels License

Table 1: images used, with source and license. Full links are in [6].

To have something to measure against, every image is also degraded in a controlled way. With I the original in float and μ its mean over all pixels and channels:

$$I_{\text{deg}} = \text{clip}([\, \alpha\,(I - \mu) + \mu \,] \cdot r(x) + n,\ 0,\ 255), \quad n \sim \text{N}(0, \sigma^2)$$

with $\alpha = 0.5$ (contrast halved around the mean, a single μ so colours don't shift), $r(x)$ a horizontal ramp going linearly from 0.7 on the left border to 1.0 on the right one (to simulate uneven illumination), and $\sigma = 10$ grey levels of independent noise per channel. The noise uses `numpy.random.default_rng(42 + i)`, where i is the index of the image in alphabetical order, so every run gives the same files. Degraded images are saved as PNG, since JPEG would alter the noise. **The original images are used as the reference for the metrics on these synthetic degradations. In a real scenario a clean reference is usually not available**, which is why I also applied every method to the real originals and judged those results only visually.



Figure 1: the five images (top) and their degraded versions (bottom). Note the darker left side caused by the illumination ramp.

3. Methods

Every method takes the degraded 8-bit BGR image and returns an image of the same type. Parameters are fixed and the same for all images.

Baseline. The degraded image with no processing.

Linear contrast stretching. Values between the 1st and 99th percentile are mapped linearly to [0, 255] and the rest is clipped. I used percentiles and not min/max because with noise the min and max are just a few outlier pixels and the stretch would do almost nothing. I implemented two versions: *stretch_rgb* works on B, G and R separately, so each channel gets its own gain and the colour balance can change; *stretch_lab* works only on the L channel of CIE Lab and leaves a and b untouched, so hue and saturation are kept.

Histogram equalization (*histeq_lab*). `cv2.equalizeHist` on the L channel of Lab, a and b unchanged. Equalizing B, G, R separately would give strong false colours, so I did not consider it.

Fourth method: NLM denoising + CLAHE (*nlm_clahe*). First `cv2.fastNlMeansDenoisingColored` (h = hColor = 10, template 7, search window 21), then CLAHE on L (clip limit 2.0, 8×8 tiles). The reasoning is: global equalization uses a single curve for the whole image, so it can't deal with a scene that is dark on one side and bright on the other, and it amplifies noise in flat dark regions. CLAHE builds a curve per tile and the clip limit caps how much contrast can be added, and doing NLM first means CLAHE doesn't enhance the noise too. The cost is that denoising also smooths real fine detail. To see what each step contributes, I also ran *nlm* and *clahe* alone.

Metrics. All computed against the original: PSNR (`cv2.PSNR`), SSIM (my own implementation of Wang et al. [4]: Gaussian window 11×11, σ = 1.5, computed per channel and averaged, 5 px border excluded) and MAE. All the numbers are in `results/metrics.csv`.

4. Results

	img1		**img2**		**img3**		**img4**		**img5**		**mean**		
method	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	MAE
baseline	19.30	0.389	12.78	0.286	17.66	0.398	18.11	0.355	18.46	0.421	17.26	0.370	27.88
stretch_rgb	14.63	0.133	**15.22**	0.123	16.55	0.128	16.06	0.216	15.77	0.182	15.65	0.157	35.31
stretch_lab	15.50	0.211	14.54	0.169	16.92	0.200	18.23	0.287	18.60	0.281	16.76	0.230	30.91
histeq_lab	12.30	0.170	14.41	0.113	14.31	0.157	8.73	0.145	8.88	0.111	11.72	0.139	58.76
nlm	19.72	**0.813**	12.88	**0.636**	17.93	**0.849**	18.18	**0.511**	18.73	**0.735**	17.49	**0.709**	26.72
clahe	18.21	0.205	12.35	0.139	17.17	0.204	17.64	0.271	17.21	0.243	16.52	0.212	33.23
nlm_clahe	**21.69**	0.810	13.61	0.633	**18.77**	0.848	**19.72**	0.459	**21.47**	0.677	**19.05**	0.685	**25.61**

Table 2: PSNR (dB) and SSIM for each image and method on the synthetic degradations, plus the mean over the five images (MAE only as mean, the full values are in the CSV). Best value per column in bold.

On average only the methods that include denoising do better than the untouched degraded image: *nlm_clahe* has the best mean PSNR (19.05 dB vs 17.26) and MAE (25.61 vs 27.88), and with *nlm* it almost doubles the SSIM (0.685 and 0.709 vs 0.370). All the pure contrast methods are, on average, *worse* than doing nothing. The exception is img2, whose histogram has a very dark and a very bright mode: halving the contrast costs a lot there, so simply restoring the range gives the best PSNR (*stretch_rgb*, 15.22 dB vs 12.78).

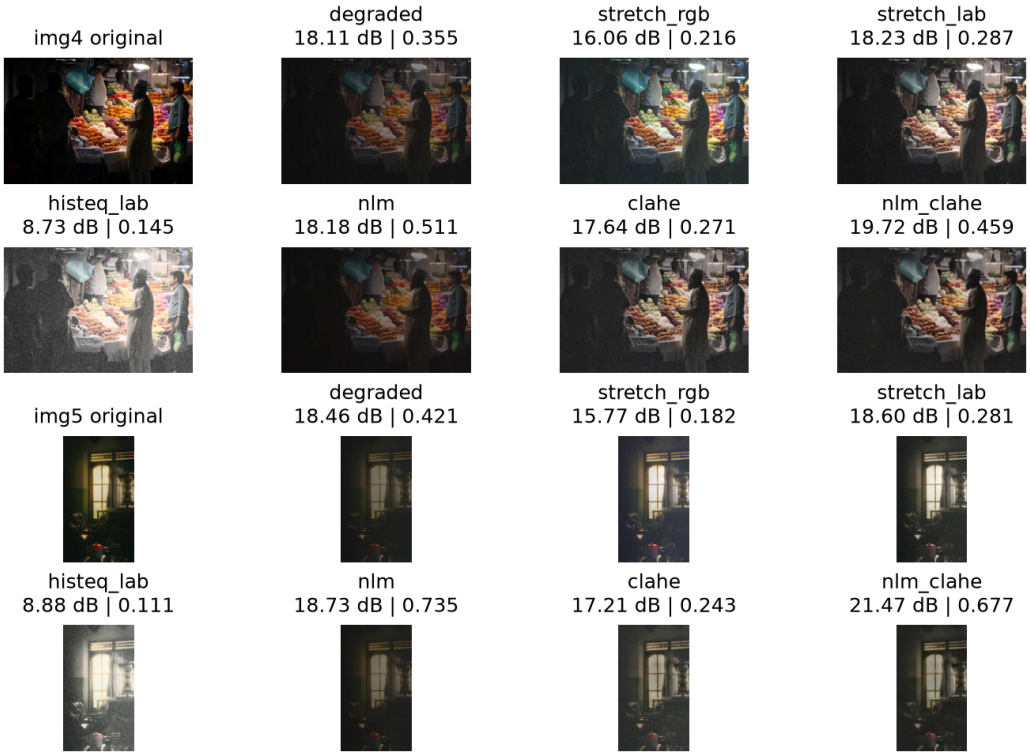


Figure 2: synthetic degradation, img4 and img5. Above each image: PSNR | SSIM with respect to the original.

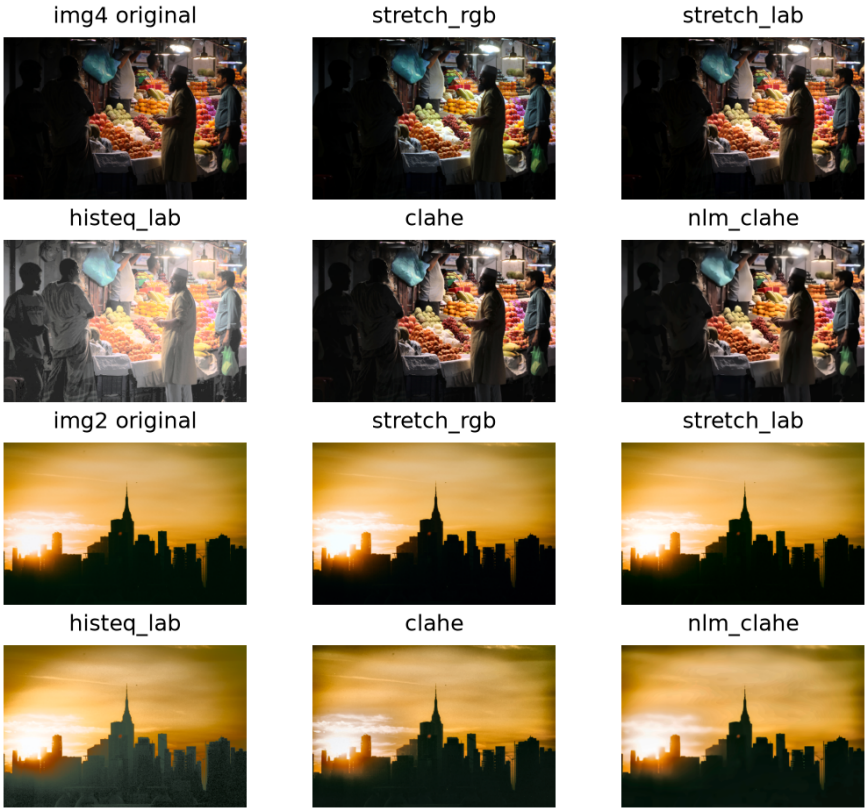


Figure 3: the same methods applied to the real originals of img4 and img2. No metrics here, since there is no clean reference.

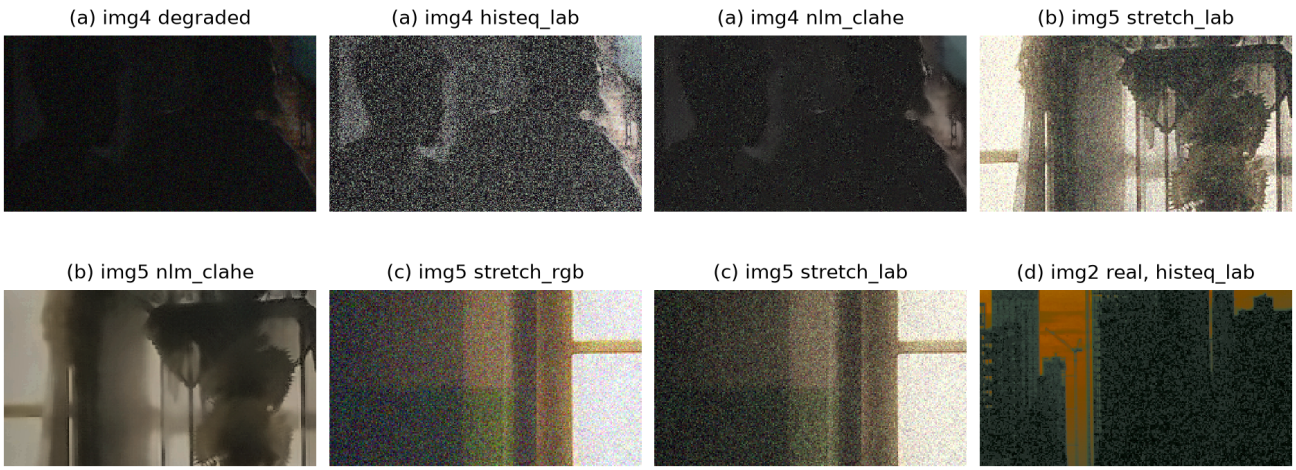


Figure 4: zoomed details. (a) shadow area of img4; (b) window decoration in img5; (c) colour of the dark wall of img5 with the two stretching variants; (d) noise and compression artifacts in the dark area of img2 after global equalization.

5. Discussion

When better numbers don't mean a better image. The clearest case is img4. Histogram equalization gets 8.73 dB, the worst value of the whole table, and yet it is the degraded result where the outlines of the two men on the left stand out most, despite the heavy noise (Fig. 4a). The reason is that the reference is itself a dark photo: any method that brightens the shadows moves away from it and PSNR punishes that, even when it adds visible information. A second case is img5: *nlm_clahe* has the best PSNR by a wide margin (21.47 dB vs 18.60 for *stretch_lab*) and a much higher SSIM (0.677 vs 0.281), but in Figure 4(b) the fine pattern of the window decoration is smoothed away, while *stretch_lab* is noisier but keeps it. Removing noise raises the metrics more than losing texture lowers them.

img4, night market. On the degraded version I prefer *nlm_clahe*: the fruit, the clothes and the people on the right are readable with little grain, and the colours stay close to the original. *histeq_lab* shows more in the shadows but is covered in noise and desaturates the scene. On the real photo, which is much cleaner, I prefer *clahe* alone: the clothes of the men on the left start to appear and the fruit keeps its colours, without the grey, flat look that global HE gives to the shadows. Global HE reveals the most, but the lamps area is washed out.

img5, dark room (synthetic). Here the colour shift of per-channel stretching is very visible: the dark wall turns blue (Fig. 4c). On the degraded image the 1st–99th percentile range is 101 levels for blue and 119 for red, so blue gets a gain of 2.5 against 2.1 for red. *stretch_lab* keeps the hue, and it is the one I prefer to keep the window details, at the price of visible grain. *histeq_lab* makes the furniture visible but turns the shadows into a grey noisy veil, and *nlm_clahe* is clean but looks painted.

img2, skyline (real image). Global HE brings out the buildings, but it turns them grey-green and amplifies noise and compression artifacts in the dark area (Fig. 4d), since that area contains few grey levels. *clahe* keeps the silhouette and adds structure to the clouds, but it creates dark borders near the frame and around the sun, and *nlm_clahe* adds visible banding in the sky. Here the stretching methods change little, because the image already covers almost the full range.

Trade-offs and limitations. NLM with h = 10 matches the noise level, but the image signal was halved by $\alpha = 0.5$, so relative to the content the filter is strong and oversmooths textures. A smaller h would keep more detail and leave more noise. I kept one value for every image, as the assignment asks. Global methods cannot compensate the illumination ramp, because they apply the same curve to every pixel; only CLAHE can, and only partly. Percentile stretching clips light sources (lamps, the sun) to white. Finally, the metrics measure fidelity to a dark reference, not visibility, and five images with a single degradation level is a small test set.

Conclusion. No method is best everywhere. On noisy inputs, NLM + CLAHE gave the best metrics and a reasonable visual compromise, at the price of fine texture. On clean real photos, CLAHE alone was the most useful. Per-channel stretching should be avoided because of the colour shifts, and global HE needs denoising first or it amplifies noise and compression artifacts.

References

[1] OpenCV documentation, https://docs.opencv.org (equalizeHist, createCLAHE, fastNlMeansDenoisingColored, PSNR).

[2] A. Buades, B. Coll, J.-M. Morel, "A non-local algorithm for image denoising", CVPR 2005.

[3] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization", Graphics Gems IV, 1994.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity", IEEE Trans. Image Processing, 13(4), 2004.

[5] R. C. Gonzalez, R. E. Woods, Digital Image Processing, 4th ed., Pearson, 2018.

[6] Images, Pexels License (https://www.pexels.com/license/):
img2: https://www.pexels.com/it-it/foto/citta-alba-tramonto-punto-di-riferimento-18927434/
img3: https://www.pexels.com/it-it/foto/strada-nebbiosa-con-alberi-spogli-in-un-autunno-nebbioso-34649347/
img4: https://www.pexels.com/it-it/foto/vivace-mercato-notturno-con-prodotti-freschi-34559646/
img5: https://www.pexels.com/it-it/foto/luce-del-sole-30422461/